pfi

Python for Fixed Income

COURSE 8 · IMPLEMENTING AGENTIC AI

What an Agent Is

WEEK 1 · DAY 1

Created by **Jeff Lenamon**

This course is educational content created in a personal capacity. Nothing here is investment advice or a recommendation to buy or sell any security. All portfolio data is synthetic.

Today: what an agent is

The week's bond (synthetic)

Coupon	5.000%, semiannual, 30/360
Settles	2026-10-02
Matures	2031-10-02
Yield	5.060%
Saved run	recorded 2026-10-08

Today

1. A model, tools, a loop
2. The key, kept out of Git
3. A request on the wire, and replay
4. A tool is a function plus a description
5. The first tool request, and its settings
6. The tool run, drawn as components
7. Reading the answer; what it cost
8. Workflows and agents

Every model response in the notebook is a saved response, replayed. A live run can differ.

The rules for agents

What an agent produces: text, every number from a tool

Allowed names

the agent returned

the saved run

the brief states

Never called

the agent's opinion or view

what the agent believes

the agent's advice

On the synthetic book (from day 3): a description, never a recommendation

Allowed names

DV01 per 100 of face

position DV01 in dollars

the largest DTS contributions in the file

Never called

cheap, rich, or attractive

a trade idea or a recommendation

overweight or underweight

How an agent works

The problem	A model answers from training: it cannot look anything up on your machine or compute a DV01 exactly
The tools	Your own functions, described to the model as a list it can ask for
The loop	Send, read the reply, run a requested tool, add its result, send again; stop at text
You control	The instructions, which tools it may ask for, and a step limit
What comes out	The final text and the list of messages: the record of every step

A model, tools, a loop

The model

runs on the provider's servers

receives messages and tools

returns text, or tool requests

runs nothing on your machine

The tools

runs on your machine

your functions, described

run only when your code runs them

today: bondmath's dv01

The loop

runs on your code

send, run tools, send back

stop at text or a step limit

today: two requests by hand

Only the model is remote. Every number a tool returns is computed by your own code.

The five components

Component	In this course	Lives in	Built on
model	gpt-4.1-mini, named in agentkit.MODELS	the provider's servers	day 1
instructions	agentkit.DESK_PROMPT, the system message	your code, first message	day 1
tools	functions with a tool_schema each	your code	days 1 and 3
memory	the list of messages, resent in full	your code, then your disk	days 2 and 12
plan	the order of the model's tool requests	the trace	days 2 and 11

Nothing here is a separate product: a framework holds the same five things for you.

The key and the mode

get_secret looks in order

1. the environment
2. Colab's Secrets panel
3. my_bondmath\.env

Before anything is sent

```
git check-ignore -v .env cache/agents/

.gitignore:4:.env    .env

.gitignore:5:cache/    cache/agents/
```

```
OpenAI key loaded: False. Mode: replay.
```

The key cell is the only cell that touches the key, and it prints only whether one loaded.

How the key of a request works

The problem	A notebook must print the same text each run; a hosted model does not promise that
The key	The SHA-256 of the method, path, and body with keys sorted: one name per request
Replay	Returns the saved response with that name, or raises LookupError
You control	The mode (a key in the environment means live) and any word of a request
What comes out	The response, and client.exchanges: every exchange, in order

A request on the wire

```
POST /v1/chat/completions
Authorization: (the key, a header)
{
  "model": "gpt-4.1-mini",
  "messages": [...], "tools": [...],
  "temperature": 0, "seed": 42, ...
}
```

exchange_key: method, path, body

```
"the DV01 ..."   1d42e20fa5c722d7...

"the dv01 ..."   6e361ffc2a182cbf...
```

keys sorted, no spaces; never a header

4 saved exchanges copied for day 1

replay

no key: the saved response, or LookupError

live

your key: sent, then saved under its key

Each request carries every reply before it: a saved run fixes the whole path.

How a tool schema works

The problem	A model reads and writes text only, so a tool must be offered as text
The reading	tool_schema reads the function's name, type hints, and docstring
The schema	A name, a sentence on the result, and a type and a line per parameter
You control	The name, the first paragraph, every Args line: the only place a coupon is in percent
What comes out	A dict inside the request body: change it and the request changes

A tool is a function plus a description

```
def dv01(settlement: str, maturity: str,
         coupon_pct: float, yield_pct: float):
    """The DV01 in price points per 100 of
    face: the price change for a 1 bp fall.
    Args:
        coupon_pct: the annual coupon in
            percent, 5.0 for a 5% coupon.
```

What the model receives

name	the function's name: dv01
description	the docstring's first paragraph
parameters	type hints + Args: lines
required	every parameter
strict	true

`dv01("2026-10-02", "2031-10-02", 5.0, 5.06) returns {'dv01_per_100': 0.043625}`

The docstring is the prompt: the model reads it, and nothing else of your code.

How a tool request works

The problem	The model needs a result that only your machine can compute
The reply	No text: a list of tool requests, each an id, a name, and arguments as JSON text
Nothing ran	The arguments are words the model wrote against the schema
You control	Which tools are offered, parallel_tool_calls, and tool_choice
What comes out	The id and the arguments your code must run and send back

The first tool request

Sent

system: DESK_PROMPT

user: the question, with its date

tools: [dv01]

parallel_tool_calls: false

Returned in the saved run

finish_reason	tool_calls
content	None: no text at all
name	dv01
id	call_ and 24 more characters

```
arguments the model wrote: {"settlement":"2026-10-02", "maturity":"2031-10-02", "coupon_pct":5, "yield_pct":5.06}
```

Model gpt-4.1-mini-2025-04-14, temperature 0, seed 42; 1 request, 432 prompt and 42 completion tokens; saved exchange recorded 2026-10-08; replayed (no key loaded).

Nothing has run yet. The DV01 does not exist until your code runs the tool.

Every setting of a request

Setting	What it controls	What the course does
`model`	which model answers	gpt-4.1-mini, every request
`messages`	the conversation so far, system message first	built by hand today, by agent_loop from day 2
`tools`	the tool schemas the model may ask for	one tool today, more from day 3
`tool_choice`	auto, none, required, or one named tool	left to the provider
`parallel_tool_calls`	several tool requests in one reply	false, so tools run one at a time
`temperature`	how much the sampling varies; 0 the least	0, with Course 7 day 11's caveat
`top_p`	sampling from the smallest likely set of words	left alone: temperature is enough
`max_tokens`	a cap on the reply's length	300 today
`seed`	asks the provider to repeat its sampling	42, a request and no promise
`response_format`	text, JSON, or a JSON schema for the reply	from day 4, for structured answers
`store`	keep the exchange on the provider's side	the framework is told false from day 9
`n`	how many replies to return	not used: one reply, one path

41 parameters in the installed package: 2 required, 35 optional, 4 transport options

Running the tool, and the result back

1. Run it

dv01(**json.loads(arguments))

2. tool_message

the id, and the result as JSON

3. Send again

reply and result added

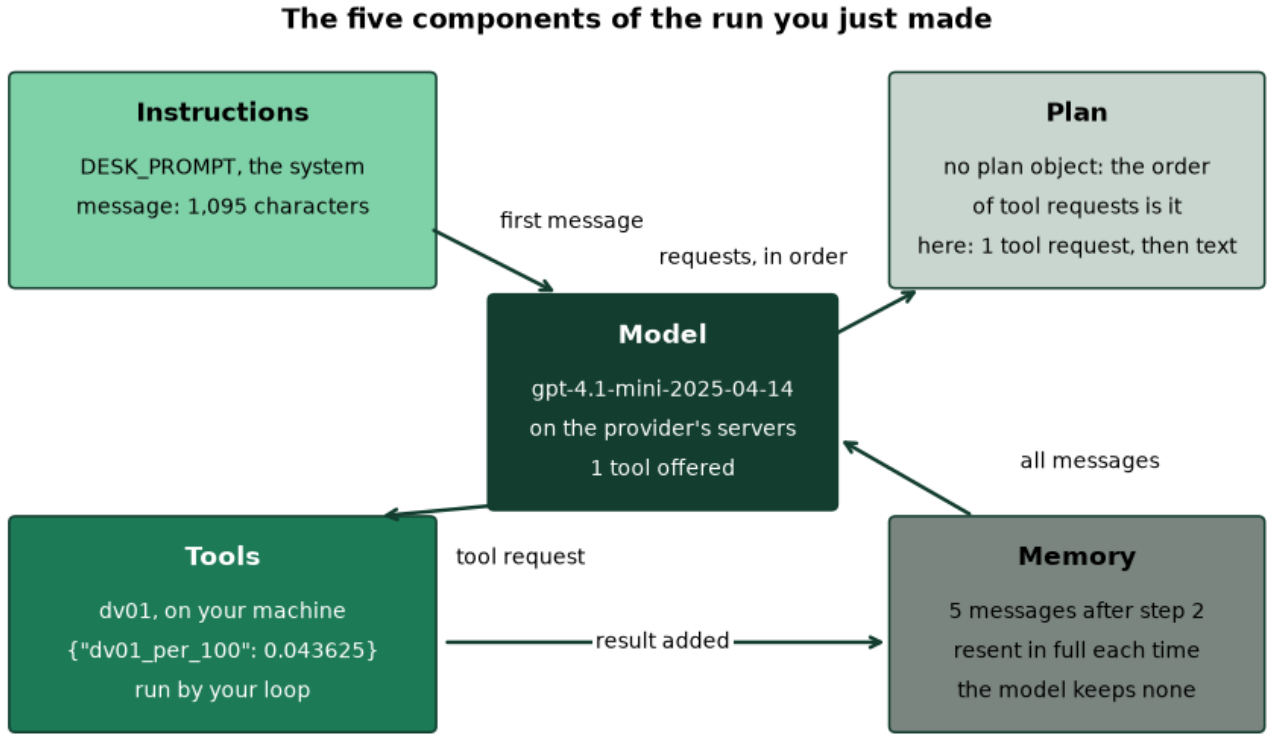
What the agent did, step by step

```
1  assistant  dv01, the arguments; tool_calls
1  tool  {'dv01_per_100': 0.043625}
2  assistant  the text; stop
```

Model gpt-4.1-mini-2025-04-14, temperature 0, seed 42; 2 requests, 952 prompt and 124 completion tokens; saved run recorded 2026-10-08; replayed (no key loaded).

The model holds nothing between requests: each one carries the whole conversation.

The run, drawn as its components



Read from the run

the model id the response returned

the system message's length

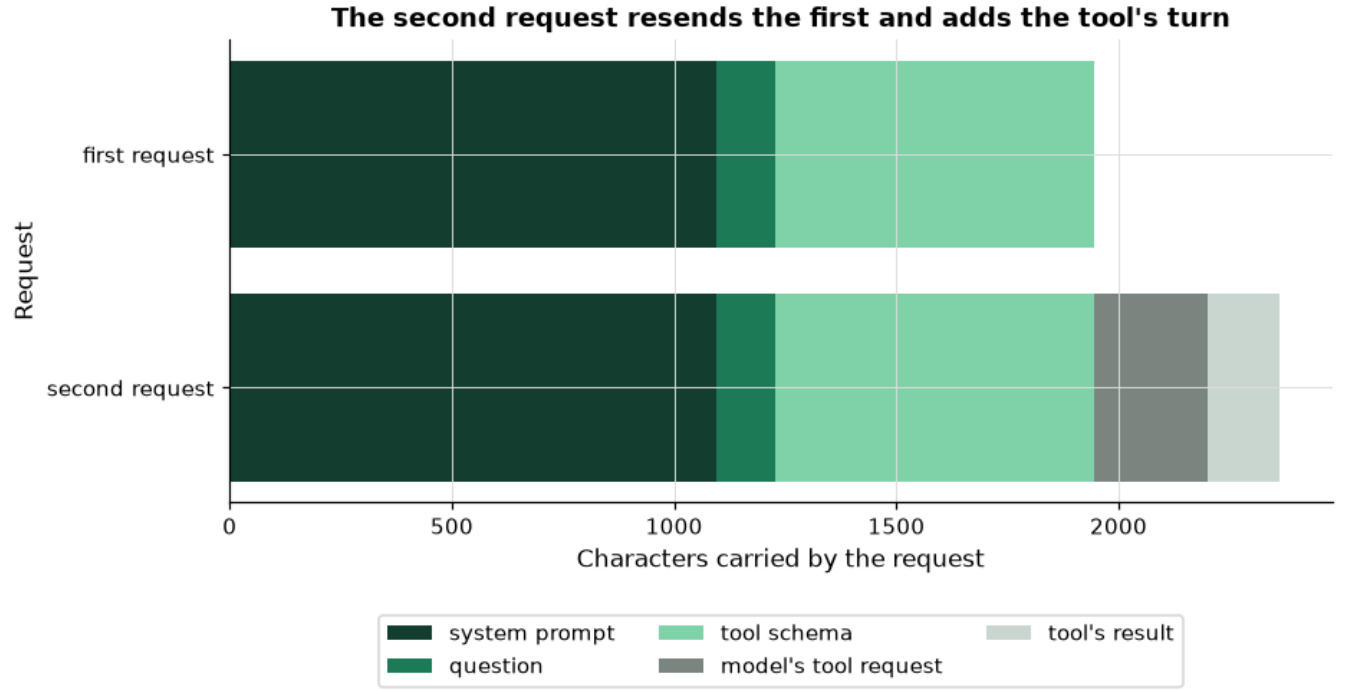
the tool's result

5 messages after step 2

the plan: one tool request, then text

An agent has no part you cannot print.

What each request carries

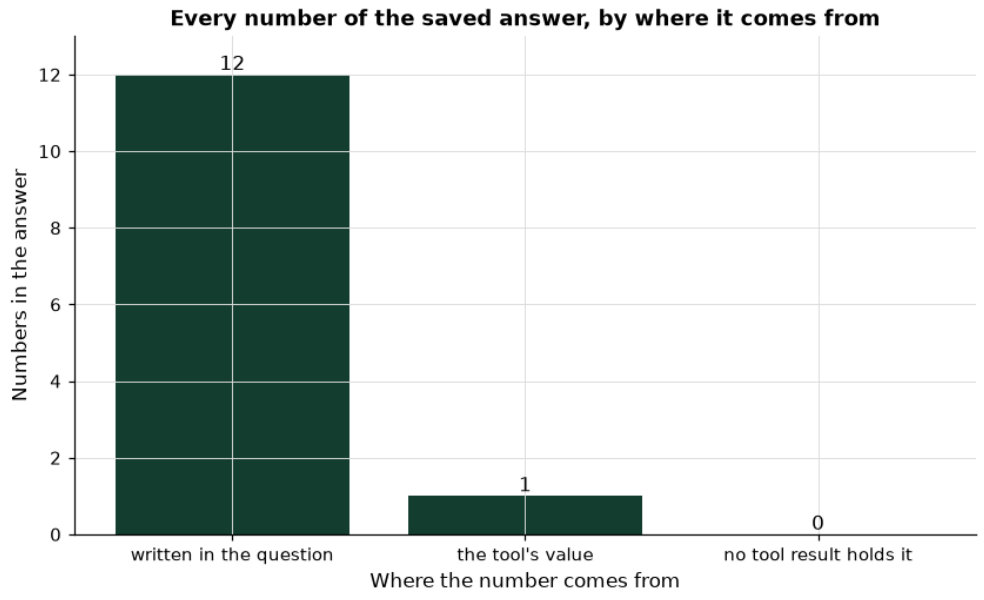


The model holds nothing between requests: every request carries the whole conversation.

How the first check of an answer works

The problem	A free-text answer can state a number no tool returned, and nothing in the text says which
The check	A regular expression finds every number in the answer
The sort	Written in the question, the tool's value at its own decimals, or neither
You control	What counts as a source, and how the decimals are compared
What comes out	A table of the answer's numbers with their source; read the neither rows first

Reading the answer



13 numbers in the answer

A first check, with two gaps:

It cannot say which tool request a number came from.

Another value's rounding could match by chance.

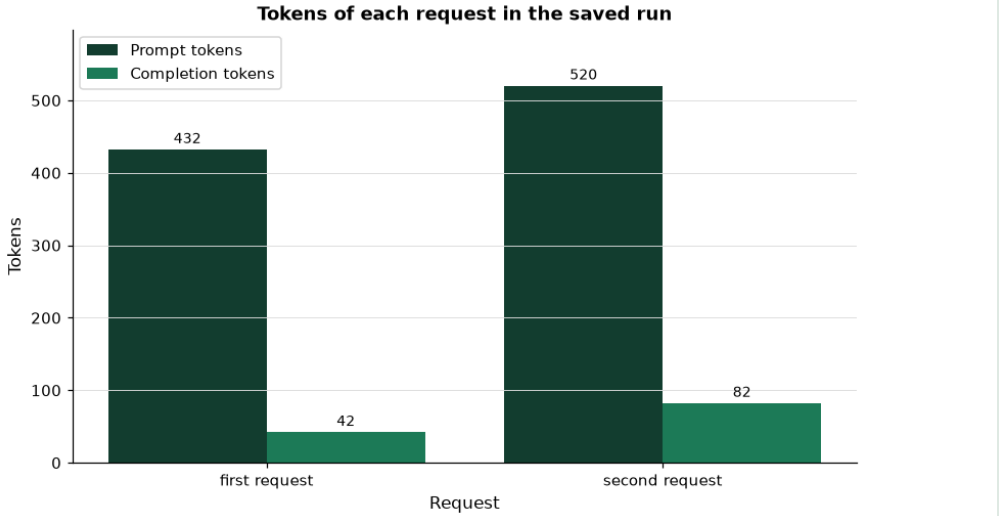
12 from the question, 1 from the tool, 0 from neither.

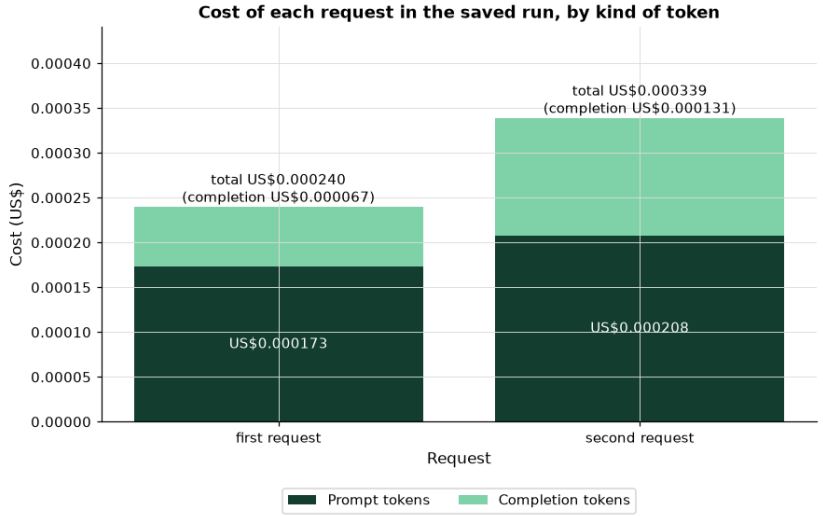
Day 4: every figure names its tool request, and check_figures rejects one that does not trace.

How a run is priced

The problem	A hosted model bills by the token, and an agent sends many requests
The usage	Each response reports prompt tokens (all it was sent) and completion tokens (what it wrote)
The cost	Each count times its price per token, added
You control	The number of requests, what every request resends, and the model
What comes out	US dollars per request and per run, and a prompt that grows at every step

What it cost





gpt-4.1-mini list price: US$0.40 per million prompt tokens, US$1.60 per million completion tokens

The saved run: US$0.000579, of which prompt tokens 65.7%. A thousand runs like it: US$0.58.

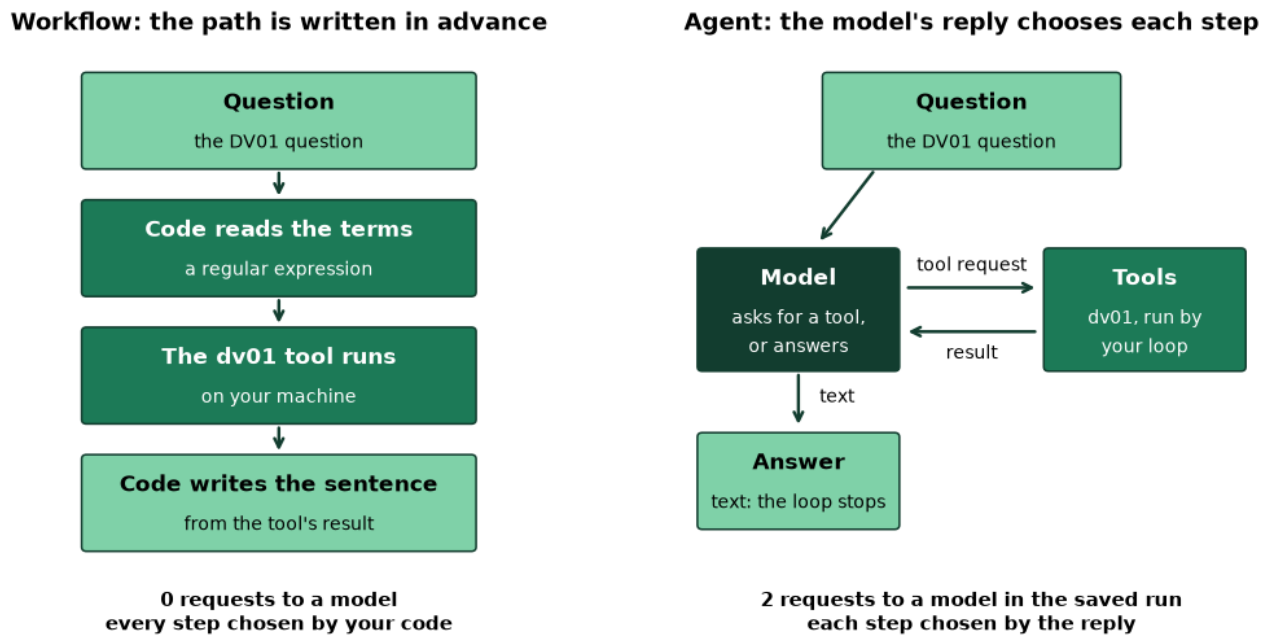
The second request resends everything the first held: every step costs more prompt tokens.

Source, read 8 October 2026: developers.openai.com/api/docs/pricing

How a workflow differs from an agent

The problem	Both use tools; they differ in who chooses the next step
A workflow	Your code chooses: steps and order written in advance, a model fills at most one step
An agent	The model's reply chooses each step; your loop runs it, within a step limit
You control	How much of the path you write down: more fixed, cheaper and more repeatable
What comes out	An answer and a trace either way; what differs is who wrote the path

A workflow and an agent



The DV01 is 0.043625 per 100 of face. The same number from both designs.

Same task, same tool, same number. The difference is who wrote the path.

Workflow or agent

Situation	Choose	Why
Every run follows the same steps	workflow	the path is written once; one trace per run
The input arrives in a fixed format	workflow	code reads it exactly, for free
The same output is needed on every run	workflow	a model's reply can vary even at temperature 0
The questions are open: tools and order vary	agent	the next step depends on the last result
The wording of the input varies	agent	the model writes arguments from any wording
Fixed steps, one step needs reading	workflow + model step	days 9 to 11 combine them

Other frameworks' words

Idea	This course	LangGraph	CrewAI
one step that does work	a model request or a tool run	node	task
the order of the steps	the lines of your workflow function	edge	process: sequential
the next step, by rule or reply	the loop's check; a router	conditional edge	process: hierarchical
what the next step can see	the messages list; later a file	state	memory
a model with instructions and tools	the loop of day 2; the Agent of day 9	node with a model	agent (role, goal, tools)
the group that does the whole job	an orchestrator and specialists	the compiled graph	crew

A reading aid, not a feature comparison. The course teaches the raw loop and the OpenAI Agents SDK.

Excel and Git

Excel side by side (tested in Excel)

`=PRICE(DATE(2026,10,2),DATE(2031,10,2),0.05,0.0506,100,2,0)`	99.73784463676976
`=MDURATION(DATE(2026,10,2),DATE(2031,10,2),0.05,0.0506,2,0)`	4.373954476118345
`=B3*(B2+B4)/10000  (DV01 per 100)`	0.043624879198739516
`=B2*C2+B3*C3  (the run's cost)`	0.0005792
`=SUMPRODUCT(B2:B3,D2:D3)  (prices as text)`	0

```
!! __pycache__/
!! cache/
```

One commit: four earlier modules beside agentkit, 7 tests. cache/ marked !!, out of Git.

The AI check

The docstring is the prompt. The agent code that comes back runs and looks right.

1

Docstring

Units, conventions, and what the function must never do: what you asked for.

2

Code back

Written for the course in the style of an AI answer. Read it.

3

Test first

Assert what the docstring promises, on a fake server. Run it: it fails.

4

Fix

Find the rule it broke, fix it, run the test again.

Today: the Args: line says a decimal, so the model sends 0.05 and gets 0.038041.

Code that runs is not the same as code that is right.

What you can do now

- Name the five components of an agent
- Keep the key in .env, out of Git, never printed
- Turn a function into a tool: tool_schema
- Send a tool request by hand, send the result back
- Choose a workflow or an agent; price the run

Week 1, Day 2 The agent loop by hand